

UNIVERSIDAD TECNOLÓGICA DE TEHUACÁN

Nombre del alumno:

ANGEL GASPAR REYES

8A

TSU EN TECNOLOGIAS DE LA
INFORMACION: DESARROLLO DE
SOFTWARE Y MULTIPLATAFORMA

TEHUACÁN, PUEBLA, 22 de Marzo de 2026

1. ¿Qué es una multisesión?

Una **multisesión** ocurre cuando un mismo usuario tiene **más de una sesión activa al mismo tiempo**.

Ejemplos reales:

- Abre tu sistema en **dos pestañas** del navegador.
- Inicia sesión en **celular y laptop** al mismo tiempo.
- Usa modo incógnito + navegador normal.

Cada acceso genera una **sesión independiente**, aunque sea el mismo usuario.

2. Problemas con sesiones concurrentes

Aquí es donde muchos sistemas fallan:

Inconsistencias de estado

- Un usuario cierra sesión en una pestaña... pero sigue activo en otra.
- Cambia contraseña → otras sesiones siguen válidas.

Conflictos de datos

- Edita algo en dos pestañas → se sobrescriben datos.
- Acciones duplicadas (ej: pagos, registros).

Seguridad

- Sesiones robadas (session hijacking).
- Sesiones abiertas en dispositivos olvidados.

3. ¿Cómo se invalida una sesión?

Hay varias formas profesionales:

1. Expiración por tiempo

- La sesión caduca después de X minutos.

Ejemplo: 30 minutos de inactividad

2. Logout manual

- El usuario cierra sesión → se elimina el token/sesión.

3. Revocación desde servidor

- Se invalida una sesión específica o todas:

Ejemplo: “Cerrar sesión en todos los dispositivos”

4. Cambio de credenciales

- Cambia contraseña → invalidas todas las sesiones activas.

4. Riesgos si no se controla bien

Si ignoras esto, pasan cosas serias:

- Acceso no autorizado (alguien sigue dentro sin permiso)
- Estados desincronizados
- Bugs difíciles de detectar
- Problemas legales (datos expuestos)

5. ¿Cómo implementarlo? (lo importante para tu entrega)

Te lo explico **simple pero profesional**, ideal para Laravel o backend moderno.

A. Estructura básica

Debes manejar sesiones con:

- **ID de sesión o token**
- **Usuario asociado**
- **Fecha de expiración**
- **Estado (activa/inactiva)**

Ejemplo de tabla:

sesiones

- id
- user_id
- token
- fecha_expiracion
- activo
- dispositivo (opcional)

B. Permitir multisessiones (correctamente)

Cuando el usuario inicia sesión:

Generas **una nueva sesión**

NO eliminas las anteriores (a menos que quieras controlarlas)

C. Validación en cada request

Cada petición debe validar:

si (token existe && activo && no expirado)

 permitir acceso

sino

 rechazar

D. Control de expiración

Dos formas:

1. Expiración automática

- Revisar en cada request

2. Limpieza programada

- Cron job que elimina sesiones expiradas

E. Recuperación de sesión

Ejemplo típico:

- Usuario pierde conexión
- Vuelve → si el token sigue válido → continúa sesión

Esto es clave para UX real.

F. Control avanzado (recomendado para 10/10)

Puedes agregar:

✓ **Límite de sesiones**

Máximo 3 sesiones por usuario

✓ **Cierre remoto**

- Panel: “cerrar sesiones activas”

✓ **Detección de dispositivo**

- Guardar:
 - IP
 - Navegador
 - Ubicación aproximada

6. Tests que puedes presentar

Esto te suma puntos fuertes:

✓ **Caso 1**

- Login en 2 dispositivos → ambos funcionan

✓ **Caso 2**

- Logout en uno → el otro sigue activo

✓ **Caso 3**

- Expiración → acceso denegado

✓ **Caso 4**

- Token inválido → acceso bloqueado

Manejo de multisesiones

El sistema permite múltiples sesiones activas por usuario mediante el uso de tokens únicos almacenados en base de datos.

Cada sesión contiene:

- Identificador único
- Usuario asociado
- Fecha de expiración
- Estado de actividad

El sistema valida cada solicitud verificando:

- Existencia del token
- Estado activo
- Vigencia

Se implementa expiración automática y cierre manual de sesión.

Esto permite:

- Acceso desde múltiples dispositivos
- Persistencia ante fallos de conexión
- Mayor control y seguridad